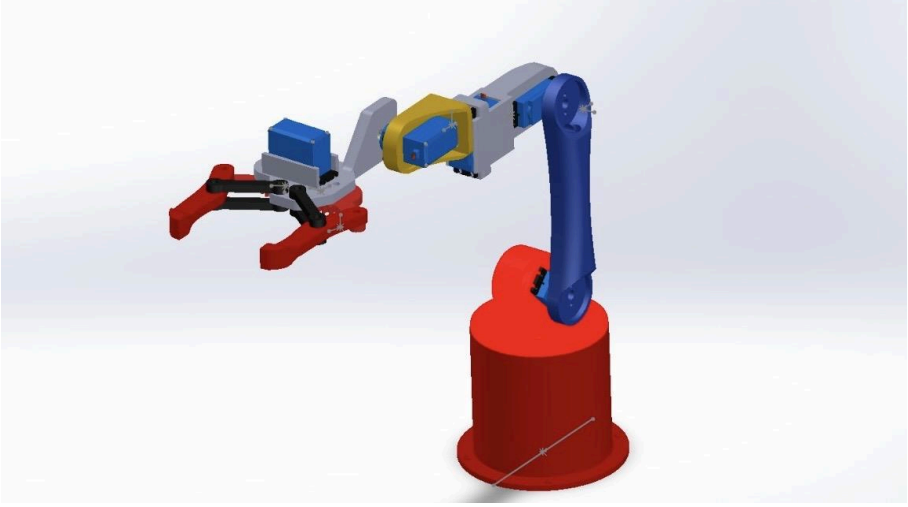
MCT342 Industrial Robotics

Milestone 2



Name	Student Code
Abdelrahman Farag	23p0137
Ahmad Hatem Mahmoud	23p0322
Basem Mohamed Fotoh	23p0005
Juliana Hany Saad ElDeen	23p0267
Mohamed Tarek Mohamed Elkholy	23p0142

Introduction **1**
MoveIt 2 Simulation Integration **2**
1. Integration Steps 2
2. Configuration Validation 2
Trajectory Planning & Execution in Simulation **2**
Motion Planning in RViz2 2
Methods and Steps **2**
Figures **4**
Issues and Resolutions **9**
Conclusion **9**

Introduction

This report documents the completion of Milestone 2, which focused on integrating a robotic arm with MoveIt 2, performing trajectory planning in simulation, and setting up the physical hardware joints. The goal was to bridge the gap between simulation and real-world actuation, ensuring the robotic arm can be controlled both virtually and physically.

MoveIt 2 Simulation Integration

1. Integration Steps

We used the MoveIt 2 Setup Assistant to integrate our robotic arm model, including the gripper. The following steps were taken:

- Loaded the URDF/Xacro model of the robotic arm into the Setup Assistant.
- Defined planning groups for the arm and gripper.
- Selected the kinematic solver (we used kdl_kinematics_plugin).
- Set joint limits based on the physical constraints of our robot.
- Configured self-collision matrix and virtual joints.
- Generated the MoveIt 2 configuration package for our robot.

2. Configuration Validation

The robot was visualized in RViz2 using the MoveIt MotionPlanning plugin. The planning groups, joint states, and collision objects were displayed correctly.

Trajectory Planning & Execution in Simulation

Motion Planning in RViz2

We performed both joint-space and Cartesian-space planning using the MoveIt 2 interface in RViz2.

- Collision-free trajectories were generated by adding collision objects to the scene.
- Joint-space planning was tested by setting target joint values.
- Cartesian planning was demonstrated by moving the end-effector to a target pose.

Methods and Steps

1. go to your workspace and source install and setup: source install/setup.bash

2. type: ros2 launch moveit_setup_assistant setup_assistant.launch.py

3. create new package and browse your urdf

4. click load files (if you didn't source, you gonna see an error here that he cant access your files)

5. change in urdf files: file:/home instead of file:///home

6. in self-collisions tab: change sampling density to max and click generate collision matrix

7. uncheck disabled if you think they gonna collide

8. in planning group, make a group to control all robot to go to specific coordinate and one for the gripper only to open and close together

9. create new group with the name from yaml file arm_group (without putting _controllers)

10. choose kdl_kinematic from kinematic solver

11. add joints: 1 -> 5 (without the last 2 of the grippers)

12. repeat last steps (9-11) with name hand_group and without kinematic solver

13. you can edit if you missed something

14. in robot pose, you can add poses like initial pose

15. check limits from this window and close it and edit the urdf angles if theyre wrong

16. add poses for arm group and another one for hand group

17. no need to add end effector

18. in ros 2 controllers: click to auto add controllers

19. in Moveit Controllers also same step 18

20. you should add author information in the end

21. in configuration files: create new folder in your_ws/src folder

22. click generate

23. if no errors -> exit setup assistant

24. go to this file from file explorer

25. colcon build the new folder only from ubunto

colcon build --symlink-install --packages-select Lab_moveit

26. to open demo: ros2 launch Lab_moveit demo.launch.py

27. it wont work

28. open launch folder in ubunto

29. touch demo_gazebo.launch.py

30. go open it from file explorer

31. fix identation and make sure file name paths are correct

32. nothing should work again if you tried opening

33. in initialization and joint_limits the numbers of values are integers not double (clear in ubunto errors)

34. change also in joint_limts file the accelerate limits to be true and give it value 5

Figures

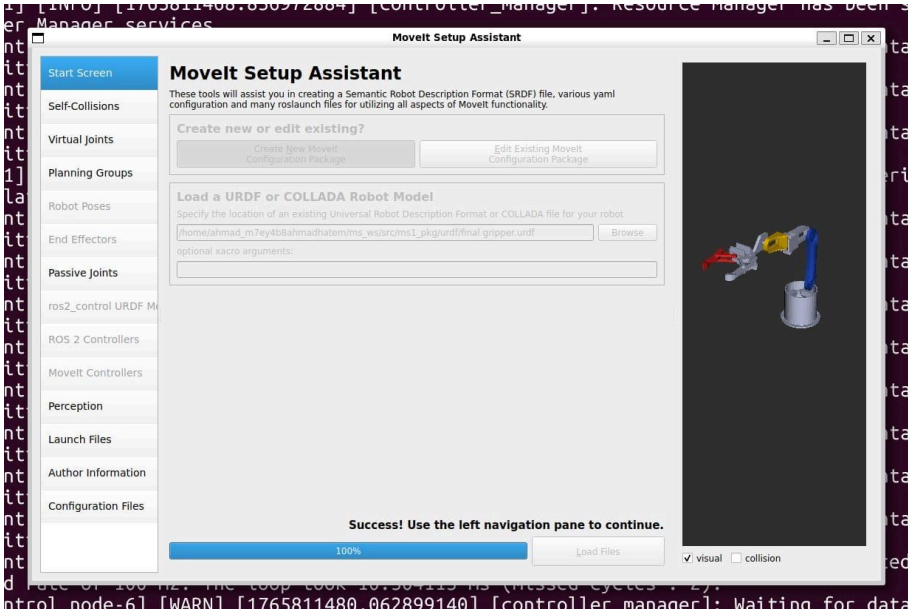


Figure 1. Loading URDF file in Moveit

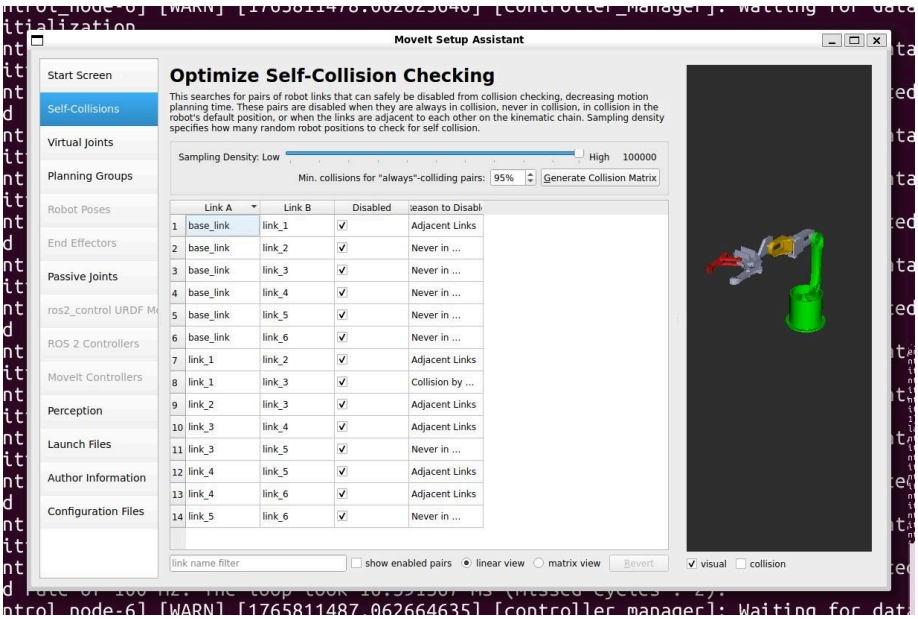


Figure 2. Changing Sampling Density to maximum, and generating the collision matrix

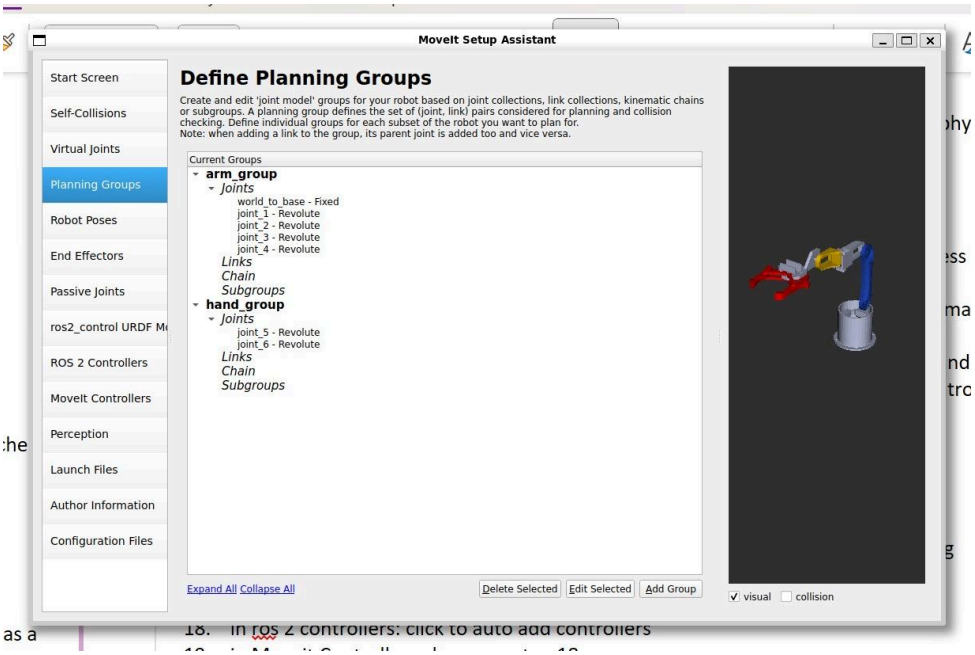


Figure 3. Creating groups for the robot (arm & end effector)

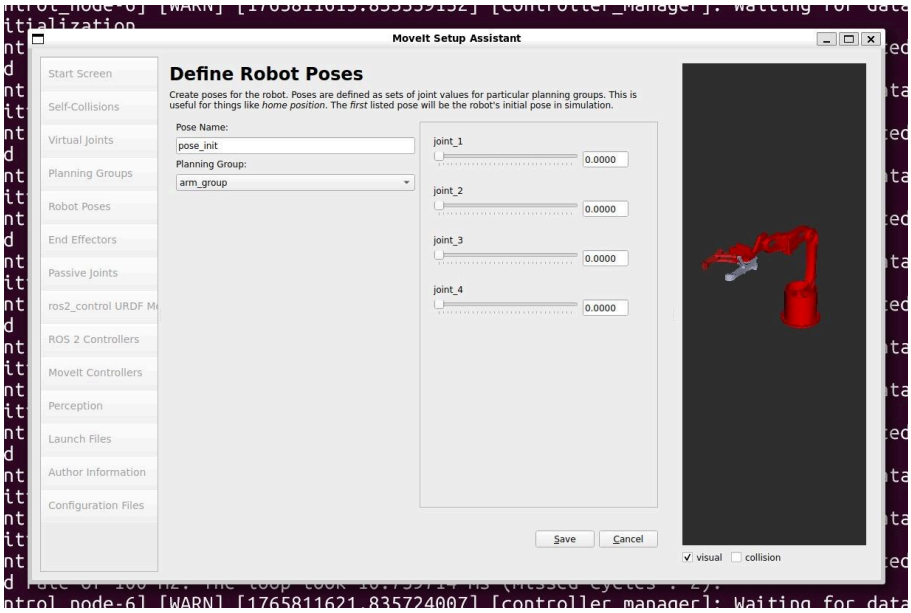


Figure 4. Creating different robot (arm) poses

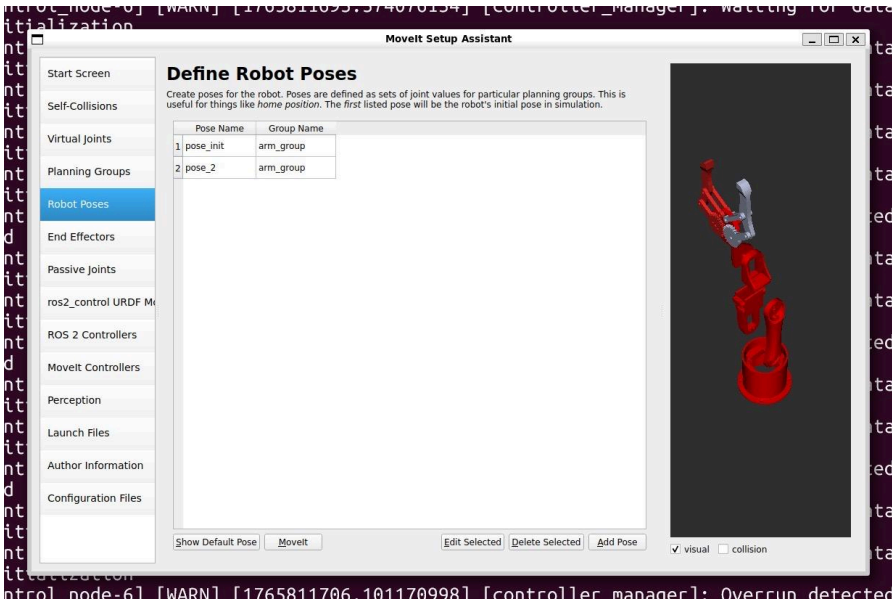


Figure 5. Creating different robot (arm) poses

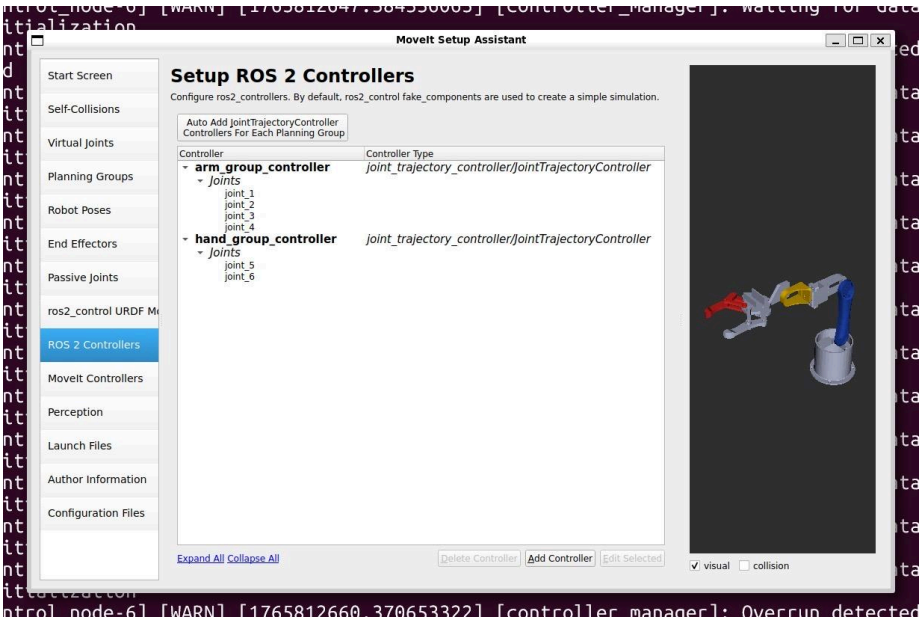


Figure 6. Setting up ROS2 Controllers

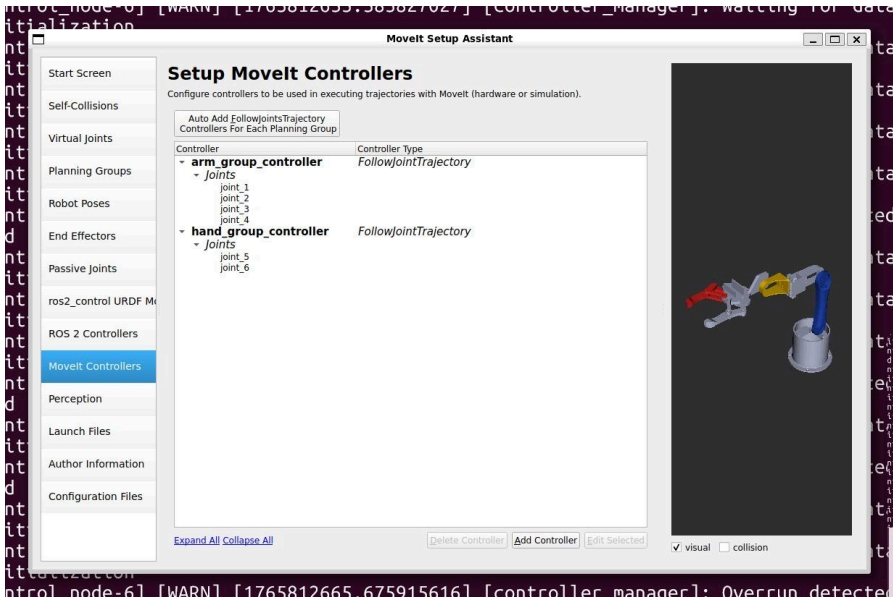


Figure 7. Setting up Moveit Controllers

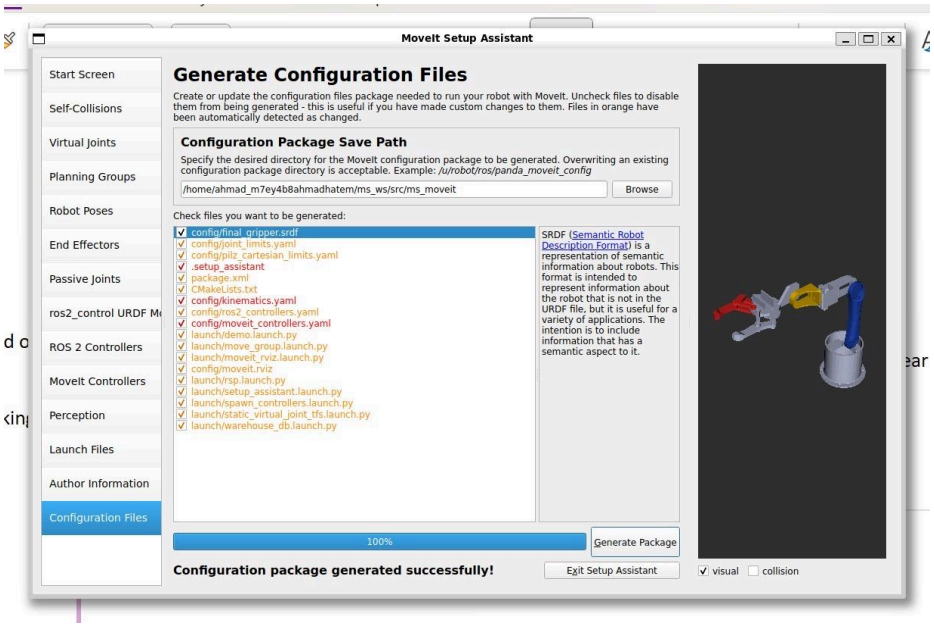


Figure 8. Generating Configuration Files

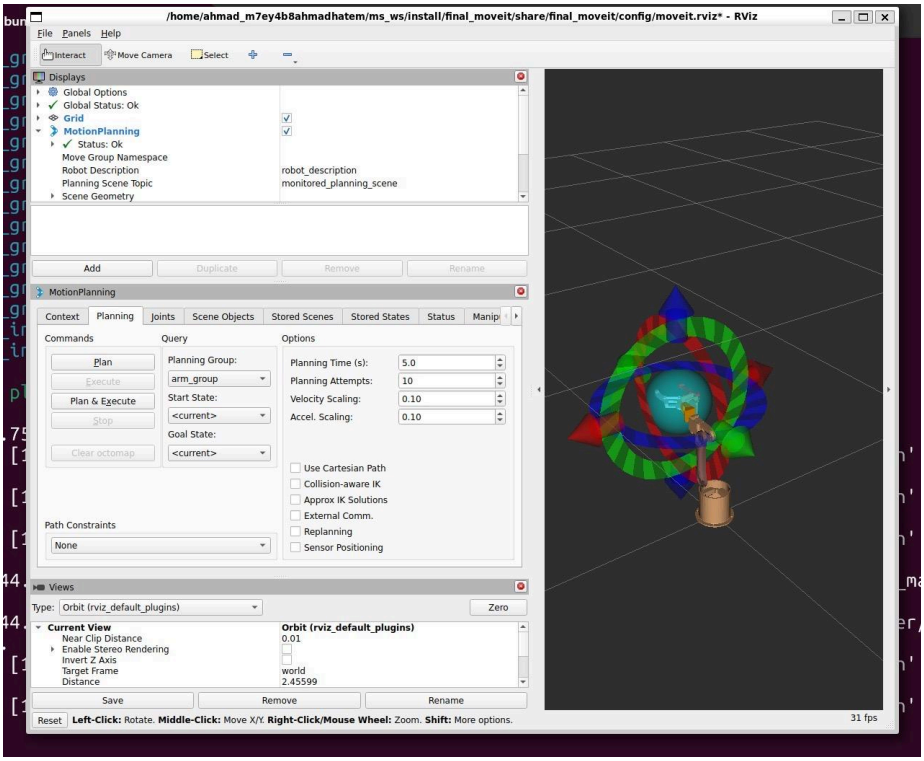


Figure 9. Opening the Robot in RViz

Issues and Resolutions

During integration, two key issues were identified and resolved. First, the gripper planning group failed to move in RViz2 due to mismatched joint names between the URDF model and the MoveIt configuration. This was corrected by standardizing the joint names in both files and updating the planning group settings in the MoveIt Setup Assistant. Second, introduced collision objects were causing trajectory planning to fail because the default collision margins were too restrictive. By adjusting the collision matrix and reducing the collision padding values in the Setup Assistant, the planner was able to successfully compute collision-free paths around obstacles while preserving safe operational clearances.

Conclusion

We successfully integrated our robotic arm with MoveIt 2, performed trajectory planning in simulation, and set up the hardware joints using an ESP32 and servo motors. The system is now capable of both simulated motion planning and physical actuation, meeting all milestone requirements.